

오늘의 작업기록 — 2026년 7월 3일

mini-ledger Layer 1 — 환불(refund) 코드 복기

(7/2 야매로 짠 환불을, 완성본 놓고 역방향으로 뜯어 온전히 이해한 날)

한눈에

항목	내용
목표	어제(7/2) 코드 먼저 받아 짠 환불을 복기로 완전히 이해
방식	완성본(커밋 85b3af6)을 놓고 검증 질문 6개 역방향 답하기
결과	6개 전부 청산✓ + 대입/게터/이름vs동작 등 근본 개념 다수 확보
오늘의 핵심	★ = 는 "같다"가 아니라 "답아라" (오늘 최대 수확)
새로 잡은 것	게터의 본질 · 이름 vs 동작 · 필드 vs 매개변수 · nextTxId/txId 관계
코드 변경	없음 (복기 세션 — 코드는 어제 커밋 상태 그대로)
의미	야매(코드 먼저)로 짚어도, 복기로 온전히 내 것 — 7/2 예외를 정직하게 메움

어제 환불을 여건상 "코드 먼저, 이해 나중"으로 갔다. 오늘 그 빚을 갚았다. 완성 코드를 한 줄씩 뜯으며 "왜 이렇게?"를 6개 질문으로 청산했고, 그 과정에서 **자바를 관통하는 대입(=)의 본질**을 처음 제대로 잡았다. 반쪽이던 7/2가 오늘로 완결됐다.

0. 출발점

- 7/2: 환불 4파일 구현 + 커밋(85b3af6). 단 여건상(SQLD CBT 병행) 잔디 우선 → 코드 먼저 받고 이해는 오늘로 미룸.
- 오늘 목표: 그 완성본을 역방향 복기 — 설계 결정 4개가 코드에 어떻게 박혔는지, 새 문법은 원지 전부 이해.
- 다음: 정산 · 복식부기 (Layer 1 나머지) — 솔로 런 복귀 (야매는 7/2 한정이었음).

PART 1 — ★ 오늘 최대 수확: = 는 "답아라"다

복기 중 이 한 줄에서 막혔고, 뚫으며 자바 전체가 열렸다:

```
Transaction original = findTransaction(txId);
```

막혔던 오해: "original = findTransaction" → "original이 findTransaction과 같다"로 읽음. (수학의 = 처럼.)

진실: = 는 등호(같음)가 아니라 화살표(←) — "오른쪽을 왼쪽에 답아라".

읽는 순서는 **오른쪽 먼저**: 1. findTransaction(txId) 가 먼저 실행 → txId번 거래를 찾아 돌려줌 2. = 가 그 결과를 왼쪽 그릇에 담음 3. original 이라는 Transaction 그릇에 그 거래가 담김

조각	뜻
Transaction	담을 그릇의 종류
original	그릇의 이름 (내가 지음)
=	"오른쪽을 왼쪽에 답아라" (같다 x)
findTransaction(txId)	실행되어 거래를 내놓는 것

→ original ≠ findTransaction. original은 findTransaction이 내놓은 결과를 담은 그릇이다. 그리고 txId는 Transaction이 아니라 findTransaction한테 주는 입력(몇 번 거래 찾을지).

이미 써온 같은 구조: Account fromAccount = getAccount(fromId); — 완전히 동일. "[그릇종류] [이름] = [실행결과]" = "실행결과를 그릇에 답아라."

비유: 자판기(findTransaction)에 번호(txId) 넣고 → 나온 음료(거래)를 → original이라는 컵에 담기. "컵과 자판기가 같다"가 아니라 "자판기가 내놓은 걸 컵에 담는다."

이거 하나로 앞으로 모든 코드가 쉬워진다. = 볼 때마다 "답아라, 오른쪽 먼저" 하면 된다.

PART 2 — 검증 질문 6개 청산 (설계 결정 4개)

Q1 — 원본 지목: 3개 파일에 뭘 추가했나

"3번 거래 환불"이 되려면 거래에 번호(id)가 필요. 세 조각:

파일	추가된 것	역할
Transaction	<code>id</code> 필드 + 생성자 매개변수 + <code>getId()</code>	거래가 번호를 갖는다
Bank (필드)	<code>nextTxId</code> 카운터	번호를 매긴다 (1,2,3...)
Bank (메서드)	<code>findTransaction(txId)</code>	번호로 찾는다

→ **찾는다** / **매긴다** / **찾는다**. (그리고 `findTransaction` 의 `t.getId() == txId` 에서 `==` 인 이유 = txId가 숫자라서. 문자열이면 equals, 숫자면 ==.)

Q2 — 환불을 왜 REFUND로? (TRANSFER 아니라)

```
history.add(new Transaction(nextTxId++, TransactionType.REFUND,
    original.getTo(), original.getFrom(), original.getAmount()));
```

환불도 "돈 이동"이라 TRANSFER로 찍으면 **일반 이체와 구분 불가**. 2 `TRANSFER b->a` 만 보면 "1번 취소한 환불"인지 "b가 a한테 보낸 별개 이체"인지 모른다. → **REFUND** **enum** 신설로 기록에 못박음. 6/27~28 원칙 **"기록 하나로 재구성 가능"**의 연장.

Q3 — 이중환불 방지 = 6/29 멍등성과 같은 패턴

멍등성 (transfer)	이중환불 (refund)
<code>Set<String> usedKeys</code>	<code>Set<Long> refundedIds</code> ← ①저장고 Set
<code>if (usedKeys.contains(key))</code>	<code>if (refundedIds.contains(txId))</code> ← ②검사(위)
<code>return;</code>	<code>return;</code> ← ③스킵
<code>... 돈 이동 ...</code>	<code>... 돈 되돌리기 ...</code> ← ④실행
<code>usedKeys.add(key);</code>	<code>refundedIds.add(txId);</code> ← ⑤저장 (아래)

→ 뼈대 판박이: **Set + contains(위) + return + add(아래)**. 막는 대상만 다름(같은 키 vs 같은 거래). **검사는 왜 위, 저장은 왜 아래**: 성공한 것만 저장해야 하니까. add를 위로 올리면 → 돈 처리 실패해도 "환불됨" 도장이 찍혀 진짜 환불이 영영 막힘. → 검증 위, 기록 아래 (6/29 순서 그대로).

Q4 — refund가 왜 txId 하나만 발나

```
public void refund(long txId) {
    Transaction original = findTransaction(txId);
    ... original.getFrom() / original.getTo() / original.getAmount() ...
}
```

원본을 찾으면 from·to·amount가 **원본 안에 다 있음** → txId만 받으면 나머지는 `original` 에서 꺼냄. (transfer는 처음 만드는 거라 4개 다 받았지만, refund는 있는 걸 되 돌리는 거라 지목번호만.)

보너스 — refund엔 왜 idempotencyKey가 없나: transfer는 "같은 요청" 구분 기준이 없어 키를 받아야 했지만, refund는 **원본 id(txId)가 그 기준**. "1번 환불"은 유일하니 txId 자체가 중복 판단 기준. → refund도 멍등성을 하되, **키 역할을 txId가 대신한다**.

Q5 — id 추가했더니 왜 Bank 여러 곳이 바뀌었나

```
// 생성자 시그니처 변경: 4개 → 5개
public Transaction(long id, TransactionType type, String from, String to, long amount)
```

→ Transaction 만드는 **모든 곳**(deposit/withdraw/transfer/refund)이 id 자리(`nextTxId++`)를 채워야 함. **시그니처(받는 형태) 변경 → 호출처 연쇄 수정**. 6/25 멍등성 키 추가 때 겪은 그 패턴. + **에러=네비게이션**: 생성자만 고치고 컴파일하면 에러가 "다음 고칠 곳"을 알려줌.

Q6 — 돈 되돌릴 때 뭘 재사용했나

```
toAccount.withdraw(original.getAmount()); // 받은 쪽에서 빼고
fromAccount.deposit(original.getAmount()); // 보낸 쪽에 돌려줌
```

→ **새 메서드 안 만들**. 원래 Account의 `withdraw` / `deposit` 재사용 (transfer가 쓰던 그것). 단 **순서가 반대**:

```
transfer(a-b): from.withdraw → to.deposit (a 빼고 b 넣고)
refund(되돌림): to.withdraw → from.deposit (b 빼고 a 넣고) ← 교차!
```

환불 = 이체를 거꾸로. 받은 쪽(to)이 되백고 보낸 쪽(from)이 되받아야 원위치. → 설계 결정 4번 "돈 되돌리기 = 기존 도구 반대로."

PART 3 — 오늘 새로 잡은 근본 개념들

복기 중 파생된, 환불을 넘어 **자바 전체에** 통하는 것들:

3-1. 게터(getter) = 값 꺼내는 문

```
public long getAmount() { // ① 문 이름 + "long 준다" 약속
    return amount;       // ② 실제로 amount를 건네는 행동
}
```

- 필드가 `private` 이라 바깥에서 `t.amount` 직접 접근 불가 → `t.getAmount()` 로 꺼냄.
- ①(선언)만으론 안 됨 — `return` 이 있어야 "뭘 줄지" 정해진다. 이름(getAmount)은 라벨일 뿐, **실제로 뭘 주는지는 return이 결정.** (`return 999` 쓰면 이름 무관하게 999 나옴.)
- 반환 타입 `long` 선언 → return으로 long 안 주면 컴파일 에러. (void면 return 불필요 — deposit/withdraw가 void였던 이유.)

3-2. 이름 vs 동작 — 이름은 장식, 기호가 일한다

- `nextTxId` 의 `next` 는 아무 의미 없는 글자. IntelliJ·자바 둘 다 "다음"으로 이해 안 함. `banana` 로 바꿔도 작동 동일.
- "다음 번호"라는 동작은 이름이 아니라 `++` 가 만든다.
- `next` 를 붙인 건 순전히 **사람이 읽기 쉬우라고**(문서화). → **이름=사람용 라벨, 기호/명령=진짜 동작.** (게터의 이름 vs return과 같은 원리.)

3-3. 필드 vs 매개변수 — nextTxId와 txId는 남남

	nextTxId	txId
정체	필드 (Bank가 보관)	매개변수 (잠깐 받음)
위치	Bank 맨 위 필드 목록	메서드 괄호 (long txId)
왜?	다음 번호를 계속 기억	부를 때 받고 버림
→ 이름만 비슷한 별개 변수. 기억할 값=필드 / 잠깐 받는 값=매개변수.		

3-4. nextTxId / nextTxId++ / txId 관계

```
nextTxId(발행기) →[++로 뺏음]→ 거래의 id(저장) ←[txId로 조회]← refund 호출
```

- `nextTxId` = 발행기 (다음 번호 들고 있음)
- `nextTxId++` = 발행기에 증가명령 건 것 (값 쓰고 +1)
- `txId` = 찾을 때 넘기는 번호
- nextTxId와 txId는 직접 안 만남** — "거래의 id"를 다리로 연결. 값은 흐르되(발행→저장→조회) 세 변수는 별개. (refund(1) 해도 nextTxId는 안 바뀜.)
- 정의줄 `private long nextTxId = 1` 엔 `++` 없음(시작값 세팅). `++` 는 쓰는 곳(new Transaction 4곳)에 있음.

4. 산출물

- 코드 변경 없음** (복기 세션). mini-ledger는 `85b3af6` (환불) 상태 그대로.
- 이 작업기록(2026-07-03.md/pdf)이 오늘의 산출물 — backend-journey에 커밋으로 잔다.

5. 다음 작업

- 정산 — 이체에 수수료 분배. **솔로 런 복귀** (5단계 직접 몰기).
- 복식부기 — 모든 거래가 양쪽 계정을 건드리게 (Layer 1 마지막, 좀 무거움).
- (선택) 환불을 DEPOSIT/WITHDRAW까지 확장.

6. 개념 사전 (오늘)

- 대입 `=` : "오른쪽을 왼쪽에 담아라"(←). 같음(등호) 아님. 오른쪽 먼저 실행 → 결과를 왼쪽 그릇에.
- 게터: private 필드 값을 꺼내주는 public 문. 이름은 라벨, `return` 이 실제로 뭘 줄지 결정. 반환 타입 선언 시 return 필수.
- 이름 vs 동작: 변수/메서드 이름은 사람용 라벨(장식). 실제 동작은 기호(`++` , `==`)· `return` 이 한다.
- 필드 vs 매개변수: 기억해야 하면 필드(nextTxId), 잠깐 받고 버리면 매개변수(txId).
- 지역 변수: 메서드 안에서만 살다 끝나면 사라짐 (original, fromAccount 등). 필드와 구분.
- 시그니처 변경 연쇄: 생성자/메서드가 받는 형태를 바꾸면, 그걸 호출·생성하는 모든 곳이 수정돼야 함.
- 환불 역등성: refund도 이중 방지를 하되 키 역할을 원본 id(txId)가 대신.

7. 비유 모음

- `=` = **자판기→컵**: 자판기(오른쪽)에서 나온 음료를 컵(왼쪽)에 담기. 컵과 자판기가 같은 게 아님.
- 게터 = **콜라 버튼**: getAmount는 "콜라" 라벨, return amount는 실제로 콜라 나오게 배선. 라벨만 붙이고 배선 안 하면 눌러도 안 나옴.
- nextTxId vs txId = 번호표 발행기 vs 손님이 말하는 번호**: 발행기가 찍은 번호를, 손님이 조회번호로 넣어 찾음. 다리는 상자에 붙은 송장번호(id).
- 이름 = 라벨**: nextTxId를 banana로 바꿔도 작동 동일. 이름은 사람 보라고 붙인 것.

8. 검증 질문 (다음 복기용 — 답 적지 말 것)

- `Account acc = getAccount(id);` 에서 `=` 는 무슨 뜻인가? 어느 쪽이 먼저 실행되나?
- 게터에서 `return` 을 빼면 왜 안 되나? 이름만으로 값이 꺼내지지 않는 이유는?
- `nextTxId` 를 `banana` 로 바꾸면 코드가 깨지나? "다음 번호"라는 동작은 무엇이 만드나?
- `nextTxId` (필드)와 `txId` (매개변수)는 어떻게 연결되나? 직접 연결인가?
- refund에 idempotencyKey가 없어도 이중환불이 막히는 이유는?
- Transaction 생성자에 인자를 하나 더 추가하면, 무엇이 연쇄로 바뀌어야 하나?

9. 회고

어제 환불을 야매로 찼다 — 코드를 먼저 받고 이해를 미뤘다. 오늘 그 빚을 갚았고, 갚는 과정이 예상보다 값졌다. 완성 코드를 한 줄씩 대조하며 "왜?"를 캐물었더니, 환불 자체보다 **더 근본적인 것들**이 걸려 나왔다.

제일 큰 건 `=` 였다. "original = findTransaction"을 "둘이 같다"로 읽고 있었는데 — `=` 가 "같다"가 아니라 **"오른쪽을 실행해서 왼쪽 그릇에 담아라"**라는 걸 잡은 순간, 그동안 흐릿하던 수십 개 코드 줄이 한 번에 선명해졌다. `Account acc = getAccount(id)` 도, `List result = new ArrayList<>()` 도 전부 같은 구조였다. 이걸 환불 복기의 부산물이지만, 실은 오늘의 본체다.

게터가 왜 `return` 을 쓰는지, `next`가 왜 의미 없는 라벨인지, `nextTxId`와 `txId`가 왜 남남인지 — 하나씩 물고 늘어진 게 다 이런 근본으로 이어졌다. "야매로 짜서 대충 넘어간 날"이 될 뻔했는데, 복기로 **오히려 더 깊이** 판 날이 됐다.

그리고 확인했다 — 환불의 네 조각(원본 찾기·REFUND·이중환불·되돌리기)은 전부 6월에 배운 것의 재조합이었다. 새 건 거래 id와 REFUND뿐. Layer 1을 쌓아온 게 여기서 복리로 돌아왔다는 걸, 코드로 직접 확인했다. 다음(정산)은 다시 솔로 런으로 — 야매는 어제 하루로 끝이다.

한 줄: 야매로 짰 환불을 복기하다 `=` 의 본질("담아라")을 잡았다 — 반쪽이던 어제를 오늘로 완결하고, 환불을 넘어 자바 전체를 관통하는 근본을 손에 넣은 날.